Clase 0.1: Datos, bits y bases

Duración: 1.5 h · **Nivel 0 del árbol** · **Semana 1 del plan**

Todo lo que hace una computadora (un saldo, un acento, una foto, una petición HTTP) se guarda como ceros y unos. Al terminar esta clase podrás explicar qué es un bit y un byte, por qué un byte guarda de 0 a 255, y convertir números entre decimal, binario y hexadecimal a mano. Es la base de los tipos de C# (`int`, `long`, `decimal`) que programas el martes.

Cómo vas a estudiar esta clase

No vas a leerla de corrido. Está diseñada con técnicas que la ciencia del aprendizaje ha probado una y otra vez. Al final de la clase hay una sección que explica cada una.

Minuto	Qué haces	Técnica
0–5	Contestas el **examen previo** sin saber las respuestas	Pretest
5–25	Bloque 1: bits, bytes y potencias de 2	Bloque de foco (pomodoro)
25–30	Pausa. Al volver, escribes de memoria todo lo que recuerdes del bloque 1	Recuperación activa
30–55	Bloque 2: binario y conversiones	Práctica deliberada
55–60	Pausa corta, sin pantalla	Descanso real
60–80	Bloque 3: hexadecimal	Fragmentación (chunking)
80–90	Explicas en voz alta 2 minutos, haces tus tarjetas y llenas el glosario	Técnica Feynman

Regla: papel y pluma para todas las conversiones. Nada de calculadora ni de IA hasta la sección de respuestas.

Examen previo (5 min)

Contesta en papel **antes** de leer. Equivocarte aquí es parte del método: tu cerebro queda buscando la respuesta y la retiene mejor cuando la encuentra.

1. ¿Cuántos valores distintos puede guardar un byte?
2. ¿Por qué las computadoras usan binario y no decimal?
3. ¿Qué número es `1010` en binario?
4. ¿Qué significa la `FF` en el color `#FF0000`?
5. ¿Por qué un `int` de C# no puede pasar de 2,147,483,647?

Bloque 1. Bits y bytes (20 min)

El bit

Un **bit** (*binary digit*) es la unidad mínima de información: vale **0 o 1**. Físicamente es algo con dos estados: un transistor que deja o no pasar corriente, una zona magnética del disco orientada hacia un lado o el otro, un pulso de luz en una fibra.

¿Por qué dos estados y no diez? Porque distinguir "hay corriente" de "no hay" es muy confiable, aunque el voltaje varíe un poco. Distinguir diez niveles de voltaje sería frágil: cualquier ruido convertiría un 6 en un 7. La computadora usa binario por **confiabilidad**, no porque sea más fácil para nosotros.

Cuántos valores caben en n bits

Cada bit que agregas **duplica** las combinaciones posibles:

Bits	Combinaciones	Valores (sin signo)
1	2	0–1
2	4	0–3
3	8	0–7
4	16	0–15
8	256	0–255
16	65,536	0–65,535
32	4,294,967,296	0–4,294,967,295

La regla: **n bits dan 2^n combinaciones**, y como se empieza a contar en 0, el máximo es **$2^n - 1$**.

El byte

Un **byte** son **8 bits**. Es la unidad con la que la computadora direcciona la memoria: cada byte de la RAM tiene su propia dirección (lo verás el sábado en 0.2 Hardware).

8 bits → 2^8 = **256 combinaciones** → valores de **0 a 255**.

> **Para la entrevista:** "Un byte son 8 bits. Cada bit duplica las combinaciones, así que 8 bits dan 2^8 = 256 valores; como se empieza en cero, el máximo es 255."

Con signo: cuando también hay negativos

Si un byte debe guardar negativos, se reserva la mitad de las combinaciones para ellos: el rango queda de **−128 a 127**. Siguen siendo 256 valores. En C# el `byte` es de 0 a 255 y el `sbyte` de −128 a 127.

Lo mismo pasa con `int`: son 32 bits = 4,294,967,296 combinaciones, repartidas de −2,147,483,648 a **2,147,483,647**. Esa es la respuesta a la pregunta 5 del examen previo, y la razón de que un `int` se pueda desbordar (sección 3 de 10 Teoria de CSharp).

Potencias de 2 que debes saber de memoria

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^{10}	2^{16}
1	2	4	8	16	32	64	128	256	1,024	65,536

Truco para memorizarlas: no las memorices como lista; **dóblalas**. Empieza en 1 y duplica en voz alta: 1, 2, 4, 8, 16, 32, 64, 128, 256. En una semana te saldrán solas.

Ancla para 2^{10}: 1,024 está muy cerca de 1,000. Por eso "kilo" en informática a veces significa 1,000 y a veces 1,024. El estándar dice que 1 KB = 1,000 bytes y 1 KiB = 1,024 bytes, pero Windows escribe "KB" cuando en realidad son KiB.

Pausa de recuperación (5 min)

Cierra esta clase. En una hoja en blanco escribe **todo lo que recuerdes** del bloque 1, sin mirar. Después ábrela y marca en otro color lo que te faltó. Eso que te faltó es justo lo que debes repasar.

Bloque 2. Binario (25 min)

Los sistemas de numeración son posicionales

En decimal, cada posición vale 10 veces más que la de su derecha:

352 = 3 × 100 + 5 × 10 + 2 × 1

En binario es igual, pero cada posición vale **2** veces más:

Posición	7	6	5	4	3	2	1	0
Valor	128	64	32	16	8	4	2	1

El bit de la derecha es el **menos significativo** (LSB) y el de la izquierda el **más significativo** (MSB).

Imagen para recordarlo: 8 focos

Imagina 8 focos en fila. Cada foco tiene escrito su valor: **128, 64, 32, 16, 8, 4, 2, 1**. Un 1 es un foco encendido y un 0 uno apagado. El número es la **suma de los focos encendidos**.

128	64	32	16	8	4	2	1	Suma
○	○	○	○	●	●	○	●	8 + 4 + 1 = **13**
●	●	●	●	●	●	●	●	**255** (todos encendidos)
○	○	○	○	○	○	○	○	**0** (todos apagados)

Dibuja los 8 focos en tu cuaderno. Vas a usar ese dibujo toda la semana.

Binario → decimal

Suma el valor de cada posición que tenga 1.

Ejemplo: `1011` → focos 8, 4, 2, 1 → 8 + 0 + 2 + 1 = **11**

Decimal → binario: método de los focos (restas)

Enciende el foco más grande que quepa, réstalo y repite con lo que sobra.

Ejemplo con 13:

1. ¿Cabe 8 en 13? Sí → foco 8 encendido. Sobra 5.
2. ¿Cabe 4 en 5? Sí → foco 4 encendido. Sobra 1.
3. ¿Cabe 2 en 1? No → foco 2 apagado.
4. ¿Cabe 1 en 1? Sí → foco 1 encendido. Sobra 0.

Resultado: **1101**.

Decimal → binario: método de las divisiones

Divide entre 2 hasta llegar a 0 y anota los residuos. El binario son los residuos **leídos de abajo hacia arriba**.

Ejemplo con 13:

División	Cociente	Residuo
13 ÷ 2	6	**1**
6 ÷ 2	3	**0**
3 ÷ 2	1	**1**
1 ÷ 2	0	**1**

Leído de abajo hacia arriba: **1101**. Usa un método para convertir y el otro para comprobar.

Ejercicios del calendario

Convierte a binario con los dos métodos: **5, 12, 100, 200, 255**. Las respuestas están al final; no las veas hasta terminar los cinco.

Bloque 3. Hexadecimal (20 min)

Por qué existe

Leer `11001000` es cansado y fácil de equivocarse. El **hexadecimal** (base 16) escribe lo mismo en menos caracteres. Usa 16 dígitos: **0–9** y luego **A–F**.

Dec	Hex	Binario		Dec	Hex	Binario
0	0	0000		8	8	1000
1	1	0001		9	9	1001
2	2	0010		10	**A**	1010
3	3	0011		11	**B**	1011
4	4	0100		12	**C**	1100
5	5	0101		13	**D**	1101
6	6	0110		14	**E**	1110
7	7	0111		15	**F**	1111

La idea clave: $16 = 2^4$, así que **1 dígito hex = exactamente 4 bits**. A esos 4 bits se les llama **nibble** (medio byte). Por eso **1 byte = 2 dígitos hex**, siempre: de `00` a `FF`.

Truco para las letras: solo memoriza que **A = 10** y que **F = 15** es "todo encendido" (`1111`). Las de en medio salen contando: B 11, C 12, D 13, E 14.

Hex → decimal

Igual que antes, pero cada posición vale 16 veces más: 1, 16, 256, 4,096…

Ejemplo: `0x2A` → 2 × 16 + A(10) × 1 = 32 + 10 = **42**

El prefijo `0x` solo avisa "esto es hexadecimal". En C# escribes `0x2A`; en binario, `0b101010`.

Hex ↔ binario: por nibbles

No hace falta pasar por decimal: cada dígito hex se convierte en sus 4 bits, por separado.

Ejemplo: `0x2A` → 2 = `0010`, A = `1010` → `0010 1010`. Compruébalo con los focos: 32 + 8 + 2 = 42.

Ejercicios del calendario

Convierte a decimal: **0x0A, 0xFF, 0x1F4**.

Dónde vas a ver hexadecimal

Lugar	Ejemplo	Qué significa
Colores web	`#FF8000`	Rojo 255, verde 128, azul 0 → naranja. 3 bytes
GUID	`3F2504E0-4F89-11D3-9A0C-0305E82C3301`	32 dígitos hex = 16 bytes
Hash SHA-256	64 caracteres hex	32 bytes (semana 6)
Dirección MAC	`00:1A:2B:3C:4D:5E`	6 bytes (semana 3)
Errores de Windows	`0x80070005`	Código de "acceso denegado"

Y una conexión que verás en la semana 3: una **IPv4** como `192.168.1.10` son **4 bytes**. Por eso cada número va de 0 a 255.

Cierre: técnica Feynman (10 min)

1. Pon un cronómetro de **2 minutos**.
2. Explica en voz alta, como si se lo contaras a alguien que no programa: qué es un bit, qué es un byte, por qué 0–255 y cómo conviertes 13 a binario.
3. Donde te trabes o uses una palabra que no sabes definir, ahí está tu hueco. Vuelve a esa parte de la clase.
4. Grábate de vez en cuando con el celular, como pide el plan (03 Plan para huir de DiSi, regla 4).

Después pasa estos términos a tu 08 Glosario, con tu definición y un ejemplo de DiSí: **bit, byte, binario, hexadecimal**. Y si te alcanza el tiempo: nibble, LSB/MSB, base.

Técnicas de memorización y aprendizaje

Estas técnicas sirven para todo el plan, no solo para esta clase.

1. Recuperación activa: sacar, no meter

Releer da la sensación de que ya lo sabes, pero no sirve para recordar. Lo que fija la memoria es el esfuerzo de **sacar** la información sin mirar. Por eso esta clase tiene pausas de "escribe todo lo que recuerdes" y tarjetas de pregunta y respuesta.

Cómo aplicarla: cierra el material y pregúntate. Si no sale, mira la respuesta y vuelve a intentarlo más tarde.

2. Repetición espaciada: repasar justo antes de olvidar

Olvidamos rápido al principio y cada vez más lento. Un repaso corto justo cuando empiezas a olvidar reinicia la curva y la hace más plana. Cada repaso puede ir más separado del anterior.

Repaso	Cuándo	Duración
1	Al día siguiente de la clase	5 min
2	3 días después	5 min
3	1 semana después	5 min
4	3 semanas después	5 min
5	Semana 12, repaso de todo el árbol	5 min

Lo más fácil es dejar que una app calcule las fechas: el plugin **Spaced Repetition** de Obsidian lee las tarjetas de abajo directamente de esta nota. También puedes usar Anki.

3. Técnica Feynman: si no lo puedes explicar simple, no lo entiendes

Explicar en voz alta con palabras sencillas revela los huecos que la lectura esconde. Es exactamente lo que pasa en una entrevista técnica.

4. Fragmentación (chunking): bloques en vez de piezas sueltas

La memoria de trabajo maneja pocas cosas a la vez. Si agrupas, cabe más: no memorices 8 bits sueltos, memoriza **2 nibbles**. No memorices 16 combinaciones de hex, memoriza la tabla de 4 bits como un solo bloque.

5. Codificación dual: palabra + imagen

Recuerdas mejor lo que tiene palabra y dibujo. Por eso los 8 focos: dibújalos a mano, no solo los leas.

6. Anclas y mnemotecnia

Une lo nuevo a algo que ya conoces:

- **255 = FF = 11111111 = "todos los focos encendidos".**

- **$2^{10} \approx 1{,}000$:** por eso los "kilos" de informática.
- **IP = 4 bytes:** por eso cada número de una IP llega a 255.
- **Doblar** en vez de memorizar la lista de potencias de 2.

7. Interrogación elaborativa: preguntar "¿por qué?"

No te quedes en el *qué*. Pregúntate *por qué*: ¿por qué 255 y no 256? ¿Por qué 2 dígitos hex por byte? ¿Por qué la computadora usa binario? Una respuesta que sabes justificar no se olvida.

8. Práctica intercalada: mezclar tipos de ejercicio

Si haces 10 conversiones iguales seguidas, mecanizas sin pensar. Si mezclas (binario→decimal, luego hex→binario, luego decimal→hex), tu cerebro tiene que elegir el método cada vez, y eso es lo que te pedirá una entrevista.

9. Pretest: intentar antes de aprender

Equivocarte antes de estudiar (el examen previo) prepara tu cerebro para la respuesta. Funciona aunque falles todo.

10. Bloques de foco y descanso

La atención baja después de 20–25 minutos. Por eso la clase va en bloques con pausas **sin pantalla**: durante el descanso el cerebro consolida lo que acabas de ver. Y **dormir** después de estudiar es cuando más se fija la memoria; estudiar de noche y desvelarte tira el esfuerzo.

Ejercicios extra (práctica intercalada)

Mézclalos; no los hagas en orden de tipo.

1. `110010` a decimal.
2. 171 a hexadecimal.
3. `0x3C` a binario, por nibbles.
4. ¿Cuántos valores distintos caben en 12 bits?
5. ¿Cuál es el número más grande que cabe en 16 bits? (Pista: es el último puerto de red que existe; lo verás en la semana 3.)
6. `10000000` a decimal.
7. 192, el primer número de muchas IPs privadas, a binario.
8. 4,096 a hexadecimal.

Preguntas de entrevista

Contéstalas en voz alta en menos de 30 segundos cada una:

1. ¿Qué es un bit y qué es un byte?
2. ¿Por qué un byte va de 0 a 255?
3. ¿Por qué las computadoras usan binario?
4. ¿Para qué sirve el hexadecimal si la computadora solo entiende binario?
5. ¿Por qué un `int` de C# tiene un máximo de 2,147,483,647?
6. ¿Qué es un nibble y por qué un byte se escribe con 2 dígitos hex?

Tarjetas de repaso

Formato del plugin Spaced Repetition de Obsidian: pregunta a la izquierda de `::`, respuesta a la derecha.

Pregunta	Respuesta
¿Qué es un bit?	La unidad mínima de información; vale 0 o 1
¿Cuántos bits tiene un byte?	8
¿Cuántas combinaciones dan n bits?	2^n; el máximo sin signo es $2^n - 1$
¿Por qué un byte va de 0 a 255?	8 bits = 2^8 = 256 combinaciones, contando desde 0
¿Rango de un byte con signo (sbyte)?	−128 a 127
¿Por qué las computadoras usan binario?	Distinguir dos estados (hay o no hay corriente) es muy confiable frente al ruido
Valores de las 8 posiciones de un byte	128, 64, 32, 16, 8, 4, 2, 1
`1011` en decimal	11
13 en binario	1101
¿Qué es un nibble?	4 bits, medio byte; equivale a 1 dígito hexadecimal
¿Cuántos dígitos hex ocupa un byte?	2, de 00 a FF
`0xFF` en decimal	255
`0x2A` en decimal	42
¿Qué es `#FF8000`?	Un color: rojo 255, verde 128, azul 0 (naranja)
¿Cuántos bytes tiene una IPv4?	4, por eso cada número va de 0 a 255
¿Máximo de un int de C#?	2,147,483,647 (32 bits con signo)
¿1 KB son 1,000 o 1,024 bytes?	1 KB = 1,000 y 1 KiB = 1,024; Windows dice KB cuando son KiB

Respuestas para verificar

Ábrelas solo después de intentarlo.

Examen previo: 1) 256 valores (0 a 255). 2) Por confiabilidad: dos estados se distinguen bien aunque haya ruido. 3) 10. 4) Rojo al máximo (255). 5) Porque un `int` son 32 bits con signo: la mitad de las combinaciones son negativas y el máximo es $2^{31} - 1$.

Binario: 5 = `101` · 12 = `1100` · 100 = `1100100` · 200 = `11001000` · 255 = `11111111`

Hexadecimal: `0x0A` = 10 · `0xFF` = 255 · `0x1F4` = 1 × 256 + 15 × 16 + 4 = **500**

Extra: 1) 50 · 2) `0xAB` · 3) `0011 1100` · 4) 4,096 · 5) 65,535 · 6) 128 · 7) `11000000` · 8) `0x1000`